

## Experiment Setup

This benchmark suite is for evaluating the runtime performance of the **effective** library. An anonymised version of the source code of **effective** can be found in `vendor/effective-0.1.0.0`. The following Haskell effect libraries are also benchmarked for comparison: **eff**, **effectful**, **freer-simple**, **fused-effects**, **mpEff**, **heftia**, and **MTL** (with **LogicT** or **ListT** for nondeterminism).

The test cases (in `bench/Bench/`) are adapted from the benchmarking suite of **heftia-effects** with the following modifications:

1. The test of coroutine is removed because the bottleneck is in the actual computation rather than the effect framework.

We added a new group of tests called **very.deep** where there are 50 reader effects before and after the effect handler being tested. This group of tests needs a lot of memory to compile so it is disabled by default in `effective-bench.cabal`. The library **fused-effects** is disabled for **very.deep** because it fails in compilation because of running out of simplifier ticks.

2. Beside the implementations of **effective**, we also add another implementation for hand-rolled monadic code that does not use the **MonadXYZ** type-classes. In the benchmark suite of **heftia-effects**, the test cases **catch** and **local** are not implemented for libraries that do not support higher-order effects. We added these tests for all libraries by implementing **catch** and **local** as handlers.
3. The **NOINLINE** pragmas in the test cases are removed. Instead we create two benchmarks **with-o0** and **with-o2** for different optimisation levels. Note that we pass **O2** to all packages in `cabal.project`, so in the benchmark **with-o0**, it is the testing programs receiving **O0** but the libraries are still compiled with **O2**. This is probably a more realistic scenario than everything being unoptimised.
4. We separate the libraries into different compilation units to make sure that each of them receives the same amount of simplifier ticks.
5. The handlers for state and reader of **mp** internally use **IOWRef** and **unsafePerformIO** to store the state. This technique is applicable to other libraries too, so for a fair comparison we create a separate test **mp.safe** where state is handled in the usual way as state-passing functions.
6. GHC is pinned to 9.10.1 and all libraries are pinned to specific versions in `effective-bench.cabal` for reproducibility. GHC 9.8.4 was also tested. All libraries compile without modification with 9.8.4 but **mp** and **mp.safe** crash at runtime, which should be caused by GHC bugs (since they don't crash with 9.10.1).
7. The library **freer-simple-1.2.1.2** is slightly patched to compile with

GHC 9.10.1.

1. `freer-simple.cabal` is modified to allow the version of `template-haskell` that ships with GHC 9.10.1
2. Line 156 of `src/Control/Monad/Freer/Internal.hs` is changed from

```
instance (MonadBase b m, LastMember m effs)
  => MonadBase b (Eff effs) where
```

to

```
instance (Monad b, MonadBase b m, LastMember m effs)
  => MonadBase b (Eff effs) where
```

It's not clear to me why GHC 9.10.1 can't see `MonadBase b m` already implies `Monad b`. The patched version is shipped in `vendor/freer-simple-1.2.1.2`.

In the deep and very deep tests for `effective`, we use the handler combinator `++>` instead of our usual fusion combinator `|>` because `effective` is designed to work with *effect sets* that have no duplicated members, but the deep and very deep tests of this benchmark introduce duplicates of reader effects. For a fair comparison, the combinator `++>` is added to `effective`, which *appends* effects rather than *unions* effects when fusing two handlers.

## How to run

The shell script `runbench.sh` runs the benchmarks. The benchmarking framework `tasty-bench` automatically runs each test case multiple times for a target relative standard deviation of 5%.

All results are generated in the directory `results/`. The raw data are recorded in `o0-results.csv` and `o2-results.csv`. The file `results/benchmark-tables.pdf` shows the data in a more readable way.

## Results

The files in `results/` shipped with this repo were generated on an Apple M4 laptop with 24GB memory. On this machine, it took around 20 minutes to compile the tests and 10 minutes to run the tests with the very deep tests enabled (and it is much quicker when deep tests are disabled). The results are shown in this file `results/benchmark-tables.pdf`. Among all results, the following two tables are probably the most interesting, showing the average running time relative to the fastest implementation:

The dashed entries are due to the following reasons:

## Benchmark results compiled with -O2: mean time (% of fastest)

| Test case                 | eff         | effective | effective.lstg | effective.lstg | effective.naive | freer    | fused     | handrolled.listt | handrolled.logict | heftia    | mp        | mp.safe  | mtl.listt | mtl.logict | polysemy  |            |         |
|---------------------------|-------------|-----------|----------------|----------------|-----------------|----------|-----------|------------------|-------------------|-----------|-----------|----------|-----------|------------|-----------|------------|---------|
| countdown.shallow.10000   | 11272.8%    | 5723.3%   | 10333.5%       | 100.0%         | –               | 3467.9%  | 6179.5%   | 196.9%           | –                 | 197.8%    | 6809.1%   | 7473.6%  | 44141.6%  | –          | 198.5%    | 48484.9%   |         |
| countdown.deep.10000      | 11189.4%    | 5855.4%   | 6515.8%        | 100.0%         | 2614.5%         | 6747.2%  | 22900.6%  | 107021.7%        | –                 | 198.7%    | 51196.5%  | 7677.2%  | 140741.0% | –          | 199.4%    | 92008.6%   |         |
| countdown.very-deep.10000 | 11084.4%    | 5491.4%   | 6572.6%        | 100.0%         | 2694.1%         | 52638.1% | 203277.0% | –                | –                 | 197.8%    | 379462.6% | 19621.1% | 994207.2% | –          | 199.4%    | 445509.0%  |         |
| catch.shallow.10000       | 240002.0%   | 6627.6%   | 1530.0%        | 100.0%         | –               | 459.1%   | 220.2%    | 125.6%           | –                 | 126.6%    | 1677.2%   | 5196.0%  | 5189.3%   | –          | 126.0%    | 6560.0%    |         |
| catch.deep.10000          | 240944.1%   | 6708.2%   | 1084.8%        | 100.0%         | 1043.9%         | 1500.8%  | 214.2%    | 1092.1%          | –                 | 126.1%    | 20300.6%  | 5273.2%  | 5303.8%   | –          | 126.8%    | 40705.3%   |         |
| catch.very-deep.10000     | 236619.6%   | 7522.2%   | 1101.3%        | 100.0%         | 1038.4%         | 11187.8% | 262.8%    | –                | –                 | 127.6%    | 254439.7% | 5367.4%  | 5311.3%   | –          | 127.0%    | 25364.0%   |         |
| local.shallow.10000       | 15025725.9% | 21470.7%  | 7946.9%        | 100.0%         | –               | 5381.8%  | 24650.4%  | 198.9%           | –                 | 199.0%    | 36720.7%  | 31279.1% | 31307.6%  | –          | 199.3%    | 87734.8%   |         |
| local.deep.10000          | 1527969.3%  | 21526.2%  | 8119.1%        | 100.0%         | 8096.4%         | 8751.2%  | 18184.2%  | 199.0%           | –                 | 199.0%    | 154209.1% | 31602.0% | 31404.9%  | –          | 199.4%    | 20184.0%   |         |
| local.very-deep.10000     | 15337705.7% | 22854.4%  | 8146.5%        | 100.0%         | 8089.2%         | 35017.5% | 18292.1%  | –                | –                 | 141151.7% | 144392.1% | 31606.8% | 31939.3%  | –          | 141702.9% | 1128501.6% |         |
| nondet.shallow.32         | 30885.0%    | –         | 961.5%         | 100.0%         | –               | 1130.5%  | 4249.5%   | 302.8%           | 437.4%            | 304.0%    | 2524.1%   | 2158.1%  | 2169.6%   | 437.1%     | 283.1%    | 10155.8%   |         |
| nondet.deep.32            | 63964.4%    | –         | 964.1%         | 100.0%         | –               | 781.2%   | 7190.0%   | 3031.4%          | 78772.3%          | 304.9%    | 6948.2%   | 7104.9%  | 7121.1%   | 78883.8%   | 305.2%    | 10175.9%   |         |
| nondet.very-deep.32       | 691440.7%   | –         | 964.4%         | 100.0%         | –               | 1238.2%  | 709.3%    | 32054.1%         | –                 | 838732.5% | 310.9%    | 51899.2% | 48738.5%  | 48728.3%   | 836104.7% | 311.7%     | 9711.8% |

Figure 1: results/o2-time-percent.pdf

## Benchmark results compiled with -O0: mean time (% of fastest)

| Test case                 | eff       | effective | effective.lstg | effective.lstg | effective.naive | freer    | fused   | handrolled.listt | handrolled.logict | heftia   | mp       | mp.safe | mtl.listt | mtl.logict | polysemy |          |
|---------------------------|-----------|-----------|----------------|----------------|-----------------|----------|---------|------------------|-------------------|----------|----------|---------|-----------|------------|----------|----------|
| countdown.shallow.10000   | 816.3%    | 680.7%    | 1500.0%        | 100.0%         | –               | 1587.9%  | 487.1%  | 1605.3%          | –                 | 533.3%   | 747.0%   | 807.4%  | 2074.0%   | –          | 631.0%   | 1647.2%  |
| countdown.deep.10000      | 816.3%    | 685.0%    | 1606.2%        | 100.0%         | 1624.9%         | 3120.9%  | 918.1%  | 10942.9%         | –                 | 5373.7%  | 2794.0%  | 1290.1% | 4973.7%   | –          | 4905.3%  | 2680.7%  |
| countdown.very-deep.10000 | 858.8%    | 754.8%    | 1637.6%        | 100.0%         | 1615.8%         | 15401.7% | 5008.4% | 77937.1%         | –                 | 45767.8% | 18894.8% | 5979.4% | 27166.0%  | –          | 40566.8% | 11058.8% |
| catch.shallow.10000       | 166915.7% | 953.9%    | 429.1%         | 158.4%         | –               | 481.0%   | 136.2%  | 1617.0%          | –                 | 100.0%   | 490.4%   | 759.3%  | 762.0%    | –          | 114.0%   | 1306.6%  |
| catch.deep.10000          | 119124.3% | 693.3%    | 339.5%         | 111.7%         | 367.1%          | 1000.4%  | 100.0%  | 5124.0%          | –                 | 1395.4%  | 2963.7%  | 542.6%  | 545.8%    | –          | 1335.1%  | 2264.3%  |
| catch.very-deep.10000     | 124221.8% | 741.6%    | 346.8%         | 111.6%         | 373.3%          | 5358.5%  | 100.0%  | 35653.1%         | –                 | 14825.6% | 25037.9% | 538.4%  | 538.0%    | –          | 12681.1% | 13110.6% |
| local.shallow.10000       | 13306.8%  | 355.5%    | 213.7%         | 100.0%         | –               | 223.2%   | 224.4%  | 480.6%           | –                 | 111.3%   | 1597.1%  | 468.6%  | 469.7%    | –          | 117.7%   | 911.0%   |
| local.deep.10000          | 128028.5% | 360.7%    | 214.8%         | 100.0%         | 216.9%          | 553.7%   | 225.0%  | 2019.5%          | –                 | 549.4%   | 4563.5%  | 472.8%  | 471.9%    | –          | 608.1%   | 1000.7%  |
| local.very-deep.10000     | 128248.2% | 384.2%    | 217.7%         | 100.0%         | 220.0%          | 2869.0%  | 226.7%  | 12881.5%         | –                 | 3729.7%  | 27488.8% | 476.5%  | 474.8%    | –          | 4097.0%  | 9825.9%  |
| nondet.shallow.32         | 1396.5%   | –         | 208.4%         | 296.4%         | –               | 213.8%   | 295.2%  | 110.9%           | 347.6%            | 106.0%   | 382.7%   | 151.1%  | 152.5%    | 374.4%     | 125.3%   | 689.6%   |
| nondet.deep.32            | 1249.7%   | –         | 100.0%         | 135.6%         | 105.9%          | 227.0%   | 184.6%  | 249.5%           | 1889.2%           | 167.2%   | 370.5%   | 182.0%  | 180.7%    | 1877.8%    | 160.7%   | 538.8%   |
| nondet.very-deep.32       | 9006.2%   | –         | 100.0%         | 134.2%         | 106.1%          | 1251.6%  | 647.8%  | 2543.7%          | 17058.4%          | 1259.2%  | 1996.0%  | 1095.6% | 1102.2%   | 16909.9%   | 1169.7%  | 2286.9%  |

Figure 2: results/o0-time-percent.pdf

- **effective** doesn't support multi-shot handlers so it doesn't have a result for **nondet**.
- **effective.lstg** (effective with light staging) doesn't have results for shallow tests because it is exactly the same as non-staged **effective**.
- **handrolled.listt** and **mtl.listt** only have results for **nondet** because they have exactly the same results as **handrolled.logict** and **mtl.logict** for tests that do not involve nondeterminism.

## Analysis

First of all, we emphasise that the results of this experiment do not necessarily generalise to practical scenarios because the testing programs are all small artificial toy programs, and the comparison between the implementations is not strictly an apples-to-apples comparison because the libraries do not implement exactly the same API. For example, **mp** and **freer** are not libraries designed for higher-order operations, so we implement **catch** and **local** as handlers rather than re-interpretable operations for them, which gives certain advantages in these tests.

The main purpose of this experiment is to check that our implementation of **effective** has the expected performance characteristics, rather than to compare the performance of the different libraries in a comprehensive way. However, we can still draw some useful conclusions from these small tests.

The following are some observations about (different ways of using) **effective**:

- Fully staged **effective** (**effective.fstg** in the tables) are the fastest in the majority of the test cases under both 02 and 00. This is not surprising because if we inspect the generated code, it is clear that the code for **effective.fstg** is overhead-free. For example, the generated code for the **countdown** test (slightly reformatted) is

```
countdownDeep :: Int -> (Int, Int)
countdownDeep n = runIdentity (runStateT p n) where
  p = StateT (\ s ->
    if (s > 0) then
      runStateT p (s - 1)
    else
      Identity (s, s))
```

The operations **get** and **put** and the (unused) reader effects are all evaluated away at compile time.

Only for **nondet** and **catch** in 00, **effective.fstg** is not the fastest. For **nondet** we believe that this is because **effective.fstg** generates code operating on vanilla lists [a], while faster implementations use CPS-based lists. It is possible to change **effective.fstg** to generate code using CPS-based lists as well.

For **catch**, we are not sure why **effective.fstg** is slightly slower than **mtl** or **freer** under 00 while the generated code looks already optimal:

```
catchDeep :: Int -> Either () ()
catchDeep n = runIdentity (runExceptT (p n))
  where
    p :: Int -> ExceptT () Identity ()
    p m = ExceptT
      (if (m > 0) then
        case runIdentity (runExceptT (p (m - 1))) of
          Left a_a6mr -> Identity (Left ())
          Right b_a6ms -> Identity (Right b_a6ms)
      else
        Identity (Left ()))
```

- Lightly staged **effective** (**effective.lstg** in the tables) does not offer performance boost in these tests. This is because there is no non-trivial handler interaction in these tests, so handler combinators aren't in any hot path of the tests.
- Non-staged **effective** performs reasonably fast compared to other implementations of effect handlers under both 00 and 02. It is not always the fastest in the shallow tests but it does very well in the deep and very deep tests because of handler fusion and storing algebras as arrays.
- The version of **effective** with the naive encoding of programs and algebras (**effective.naive**) does better than **effective** under 02 for shallow tests.

It is because GHC is willing to do more inlining for `effective.naive`, while it almost never inlines anything related to arrays.

The following are some additional remarks about implementations other than `effective`:

- Contrary to some claims online, MTL is not slow when optimisations are on. Most of the time GHC inlines typeclass members at use sites. Indeed, in our 02 experiment MTL is quite fast in all tests except for `local.very-deep`, for which GHC gives up inlining. The real problem with MTL is inflexibility: the operations and monad transformers are tied to each other rigidly.
- The performance of `fused-effects` is similar to MTL: when inlining happens it is very fast, otherwise it is very slow. However, `fused-effects` is less inlining-friendly compared to MTL. It even exhausts the simplifier ticks of GHC for very deep tests.
- Apart from our `effective`, `effectful` is another implementation that is consistently unaffected by adding layers of reader effects. This is because the monad of `effectful` is simply a reader monad `newtype Eff es a = Eff (Env es -> IO a)` (as a result it does not support the effect of nondeterminism).
- In the tests `local` and `catch`, `freer` and `mp` are unaffected by adding layers of reader effects because `local` and `catch` are implemented as handlers rather than re-interpretable (higher-order) operations for `freer` and `mp`. Therefore when the reader handlers get to handle the program, the program has already been processed into a single-operation program. Thus the layers of reader handlers hardly affect the performance. But in the tests `countdown` and `nondet` we can see that adding layers of reader effects does affect the performance of `mp` and `freer` linearly.
- For `mp` and `mp.safe`, we observe that there is indeed a noticeable performance drop in the `countdown` test from `mp` to `mp.safe` because the safe handler of state is not tail-resumptive, while `mp` is optimised for tail-resumptive handlers.
- It seems that `eff` is unaffected by adding layers of reader effects when the handlers are single-shot, but it is affected in the `nondet` test cases (where the handler is multi-shot).